

World Models in Artificial Intelligence: A Functional Taxonomy and Survey

Abstract

The term “world model” has become one of the most widely used and least consistently defined terms in artificial intelligence. Video generation models, physics simulators, game-playing agents, and robot control systems all claim to build world models, yet they pursue fundamentally different objectives. **Problem.** No existing survey organizes the field around a functional taxonomy that separates these objectives cleanly. **Approach.** We adopt Li’s (2026) three-category functional taxonomy: Renderers (output pixels, optimize visual fidelity), Simulators (output environment states, optimize structural accuracy), and Planners (output actions, optimize decision-making). **Key findings.** For Renderers, we find a clear trajectory from offline video generation (Sora) toward real-time interactive rendering (The Matrix, GameNGen) and open-source platforms (Cosmos), while physical accuracy remains a persistent weakness. For Simulators, learned physics engines (GNS) and differentiable real-to-sim pipelines (One-Shot Real-to-Sim, PIN-WM) show that physically grounded world models are feasible, but they remain concentrated in robotics-adjacent domains. For Planners, we identify two distinct paradigms — model-based RL trained in latent imagination (DreamerV3) versus LLM-based planning (Voyager, OmniJARVIS) — with complementary strengths in data efficiency and generalization. **Implications.** We identify four open problems with specific barriers, concrete attack vectors, and difficulty estimates: long-horizon consistency, unification across categories, the efficiency-generalization trade-off, and multi-modal world modeling beyond vision.

§1 Introduction

1.1 What Is a World Model?

A world model is an AI system’s internal representation of its environment. Instead of reacting to raw sensory input directly, an agent equipped with a world model can simulate possible futures, predict the consequences of actions, and plan accordingly. The concept entered mainstream AI research through Ha and Schmidhuber (2018), who demonstrated a simple architecture combining a Variational Autoencoder (VAE, a neural network that compresses images into compact codes), a recurrent network that predicts how those codes change over time, and a controller that learns to act entirely within the model’s imagination. The agent played Atari games by first learning to predict what the next frame would look like, then learning what button to press to maximize score — all inside its own simulated environment.

1.2 Why Now?

Between 2024 and 2026, the term “world model” spread rapidly across nearly every subfield of AI. In February 2024, OpenAI released Sora, a video generation model trained on internet-scale data that exhibited emergent 3D consistency and object permanence — behaviors the authors described as “world simulation.” DeepMind followed with Genie, a model that generates interactive game-like environments from a single image. NVIDIA built Cosmos, an open-source world model platform

for physical AI applications. Meanwhile, robot learning researchers were building learned physics simulators, and game AI researchers were building agents that plan actions through imagined rollouts in latent world models. All of these projects used the phrase “world model,” but they meant fundamentally different things.

1.3 A Functional Taxonomy

On June 4, 2026, Li proposed a taxonomy that resolves this terminological confusion. Drawing on the Partially Observable Markov Decision Process (POMDP) framework — a mathematical model where an agent receives noisy observations of a hidden true state and must choose actions based on this incomplete information — Li observed that a world model can produce one of three types of output:

- **Renderer:** outputs pixels (images or video frames) for human viewing. The goal is visual fidelity.
- **Simulator:** outputs environment states (positions, velocities, physical parameters). The goal is structural accuracy.
- **Planner:** outputs actions (what the agent should do next). The goal is effective decision-making.

Li further noted an important asymmetry: mastering simulation enables both rendering (by projecting physical states into pixels) and planning (by computing action consequences), but the reverse does not hold. A model that only renders pixels has no internal understanding of the physics it depicts. A model that only selects actions may have no explicit representation of the world at all. The Simulator, Li argued, is the conceptual hub.

This survey adopts Li’s taxonomy as an organizational framework. We do not extend or challenge the taxonomy itself. Our contribution is a structured overview of the field: what each category has achieved, which papers represent the current state of the art, and where the hardest open problems lie.

1.4 Scope and Related Surveys

We focus exclusively on AI/ML approaches to world models. The cognitive neuroscience origins of the concept — hippocampal replay in rodents, predictive coding in the neocortex — are acknowledged but not reviewed here.

Several existing surveys cover adjacent territory. Zhu et al. (2024) provided a comprehensive survey of general world models organized around the question “Is Sora a world simulator?” Ding et al. (2025, ACM Computing Surveys) divided world models into those for “understanding” (representing the present) and those for “predicting” (simulating the future). Kong et al. (2025) surveyed 3D and 4D world modeling with a focus on spatial perception. Chen (2026) proposed an L1-L5 autonomy taxonomy for AI research agents, with a structural approach — adopt a clean taxonomy, survey systems within each level — similar to this paper’s methodology, though applied to a different domain (research agents rather than world models). Our survey differs from these in two respects: first, we use Li’s three-category functional taxonomy as a clean organizing principle rather than enumerating systems by domain or architecture; second, we maintain strict separation between the three output types throughout, which surfaces cross-category tensions and gaps that mixed-taxonomy surveys can obscure.

Limitations of this survey. Three limitations should be noted. First, the categories in Li’s

taxonomy are functional (based on output type), not mutually exclusive in practice. Several papers discussed here — DIAMOND, Genie, DreamerV3 — have roles in multiple categories. We classify each by its primary contribution, but the boundary cases demonstrate that the categories overlap, which itself motivates the unification problem discussed in §5.2. Second, the Simulator category is concentrated in robotics-adjacent learned simulation because the AI/ML world model literature has developed primarily in that context; applications in climate modeling, computational chemistry, and structural engineering exist but are not currently framed as “world model” research. Third, Li’s taxonomy was published as a non-peer-reviewed communication (June 4, 2026) and may evolve. We use it as an organizational convenience, not as a theoretical commitment; alternative taxonomies applied to the same literature could surface different patterns.

§2 Renderer: Video Generation as World Rendering

2.1 What Is a Renderer-Type World Model?

A Renderer-type world model takes an observation (such as one or more video frames) and optionally an action (such as a camera movement or a keyboard input), and produces future frames as output. Its performance is measured by visual quality: how realistic the generated frames look to a human viewer. The underlying physical accuracy of the generated scene is a secondary concern — what matters is whether the output is visually convincing.

This framing — video generation as world simulation — was crystallized by Sora in early 2024 and has since become one of the most active research directions in generative AI.

2.2 Foundational Work

Sora (OpenAI, 2024). Sora is a diffusion transformer trained on large-scale video data to perform text-to-video generation. The key observation in the Sora technical report was not about video quality per se, but about emergent behaviors: the model exhibited 3D consistency (objects remained coherent as the camera moved), object permanence (occluded objects persisted and reappeared correctly), and basic world interaction (a painter leaving brushstrokes on a canvas that persist across frames). These behaviors were not explicitly programmed — they emerged from training a large model to predict the next video patch. Sora established the central claim of the Renderer paradigm: that scale alone, applied to next-frame prediction, can yield internal representations that approximate a world model. Sora’s code and weights were not released.

Genie (DeepMind, ICML 2024 Best Paper). Genie addressed a key limitation of Sora: Sora generates passive video; you cannot interact with it. Genie introduced the Latent Action Model (LAM), a component that infers a discrete set of possible actions purely from observing video frame transitions, without ground-truth action labels. An 11-billion-parameter model trained on unlabeled Internet videos learns both what actions are possible in a given environment and how to generate the corresponding future frames. Given a single image prompt — a photograph, a sketch, or even a hand-drawn doodle — Genie produces an interactive environment that a user can navigate. Genie’s LAM is the key conceptual contribution: it removes the need for labeled action data, which is the primary bottleneck for training interactive world models on real-world video.

Cosmos (NVIDIA, 2025). Cosmos represents the translation of Renderer-type world models from research prototypes into engineering infrastructure. It provides a complete platform: a video data curation pipeline, pre-trained models at multiple scales (2 billion to 14 billion parameters),

video tokenizers, and post-training recipes — all under permissive open-source licenses. The Cosmos-Predict2.5 update unified Text2World, Image2World, and Video2World generation within a single flow-based architecture trained on 200 million curated video clips. Unlike Sora (closed) and Genie (no public weights), Cosmos is fully available for downstream use. This openness has made it a practical foundation for applications in robotics simulation, autonomous driving data augmentation, and synthetic training data generation.

2.3 Notable Advances

Several works have pushed the Renderer paradigm beyond passive video generation toward interactive, real-time, and geometrically controlled rendering. These works represent distinct approaches to a shared goal — turning video generators into interactive world renderers — with different trade-offs. **The Matrix** (Feng et al., 2024) prioritized infinite-horizon real-time user control at high resolution (720p, 8-16 FPS), using a Swin-DPM architecture with Stream Consistency Model acceleration. **GameNGen** (Valevski et al., ICLR 2025) prioritized visual fidelity to a specific target environment, demonstrating that a diffusion model could render playable DOOM levels at 20 FPS with quality indistinguishable from the original game. **GEN3C** (Ren et al., CVPR 2025 Highlight) prioritized geometric consistency, combining video diffusion with an explicit 3D point-cloud cache for precise camera-controlled novel-view synthesis. The trade-off pattern is clear: The Matrix optimizes for generality and horizon, GameNGen optimizes for fidelity to a known target, and GEN3C optimizes for geometric accuracy. No current system achieves all three simultaneously. These works collectively demonstrate a trajectory from offline video generators into real-time, interactive, geometrically grounded rendering engines.

2.4 Core Papers for This Chapter

1. **Sora** (Brooks, Peebles et al., OpenAI, 2024) — Established the “video generation as world simulation” paradigm. Demonstrated that scaling diffusion transformers on video data yields emergent 3D consistency and object permanence without explicit physical priors.
2. **Genie** (Bruce et al., ICML 2024 Best Paper) — Introduced the Latent Action Model, which learns a discrete action space from unlabeled video alone. Showed that a foundation world model can generate interactive, action-controllable environments from any image prompt.
3. **Cosmos** (NVIDIA Cosmos Team, 2025) — The largest open-source world model platform. Unified Text2World, Image2World, and Video2World generation in a single architecture. Represents the maturation of Renderer-type world models from research to deployable infrastructure.

§3 Simulator: Physics and Geometry as World Simulation

3.1 What Is a Simulator-Type World Model?

A Simulator-type world model outputs environment states rather than pixels. An environment state might be a set of particle positions and velocities, the 3D poses of objects in a scene, or the physical parameters (mass, friction, stiffness) that govern how those objects interact. The goal is structural accuracy: when the model predicts what happens next, its prediction must respect the laws of physics and geometry, not merely look plausible.

This is the category Li described as “the most underappreciated but most critical.” A model that accurately simulates physics can be used to render pixels (by projecting the simulated state through a graphics pipeline) or to plan actions (by computing the consequences of candidate actions within the simulation). The reverse paths — from pixels to physics or from actions to physics — are far more difficult.

3.2 Learned Physics Engines

GNS (Sanchez-Gonzalez et al., ICML 2020). The foundational work in learned simulation. GNS demonstrated that a single graph neural network architecture can replace traditional numerical solvers across multiple physical domains — fluids, rigid bodies, granular materials, and deformable objects. Particles are represented as nodes in a graph; message-passing between nodes computes particle interactions; the network outputs per-particle accelerations, which are integrated forward in time. GNS established that learning a physics simulator from data is feasible with a generic architecture, spawning a substantial follow-up literature in learned simulation. The open-source implementation remains a standard baseline.

MeshGraphNets (Pfaff et al., ICLR 2021 Spotlight) extended the graph network approach to mesh-based simulations (aerodynamics, structural mechanics, cloth), achieving one to two orders of magnitude speedup over traditional numerical solvers while adapting to varying mesh resolutions. This work demonstrated that learned simulators could handle the irregular geometries common in engineering applications.

3.3 From Observation to Simulation

The central challenge for Simulator-type world models is the real-to-sim gap: how to construct an accurate physical simulation from real-world observations. Two recent works represent the state of the art.

One-Shot Real-to-Sim (Zhu et al., RA-L 2025). This paper presents the most complete differentiable real-to-sim pipeline to date. From a single robot push against an object, the system jointly optimizes three representations: geometry (via a point-based Shape-as-Points model), appearance (via a grid-based appearance field), and physical parameters (via differentiable rigid body simulation). All three components are differentiable, so gradients flow end-to-end from simulation error back through the geometry and appearance models. The result is a simulation-ready digital twin of the observed object, built from minimal interaction data.

PIN-WM (Li et al., RSS 2025). PIN-WM addresses a practical question: can we build a physics-informed world model that works on real robots? The system combines a differentiable physics engine (NVIDIA Warp) with Gaussian Splatting for visual observation. Unlike pure video-generation approaches, PIN-WM’s internal states are physically meaningful 3D poses and velocities. Gaussian Splatting eliminates the need for manual state estimation — the model learns to map from visual observations to physical states end-to-end. The authors introduced “Digital Cousins,” a physics-aware domain randomization strategy for sim-to-real transfer. PIN-WM demonstrated robust non-prehensile manipulation (pushing, sliding objects) on physical robot hardware.

3.4 Structured Representations

An emerging theme in 2025-2026 is the use of object-centric representations that explicitly model individual objects and their interactions within a world model. This approach addresses a funda-

mental limitation of monolithic latent representations: a single compressed state vector must encode everything about a scene, making it difficult to generalize compositionally (a scene with three objects should be predictable from knowledge of how single-object and two-object scenes behave). Object-centric models factorize the world into entities with independent dynamics and pairwise interactions, which improves both sample efficiency (fewer parameters per object) and generalization (new object combinations are composed from known object models). **SOLD** (Mosbach et al., ICML 2025) was the first object-centric model-based RL system to learn entirely from pixels, using Slot Attention to parse video frames into object representations and a transformer to predict slot-level dynamics. **ContactGaussian-WM** (Wang et al., 2026) introduced a unified Gaussian representation that simultaneously supports visual rendering and collision detection, with gradients flowing through a closed-form physics engine. **FloWM** (Lillemark et al., NeurIPS 2025) incorporated Lie group equivariance — a mathematical guarantee that predictions remain consistent under rotations and translations — enabling stable long-horizon physical rollouts that respect geometric symmetries. Together, these works suggest that structured representations are a promising direction for closing the gap between monolithic latent models and the compositional nature of physical reality.

3.5 Core Papers for This Chapter

1. **GNS** (Sanchez-Gonzalez et al., ICML 2020) — The foundational learned physics simulator. Demonstrated that graph networks can replace traditional numerical solvers across fluids, rigid bodies, and deformable materials with a single architecture.
2. **One-Shot Real-to-Sim** (Zhu et al., RA-L 2025) — The most complete differentiable pipeline from real-world observation to physics simulation. Jointly optimizes geometry, appearance, and physical parameters from a single robot interaction.
3. **PIN-WM** (Li et al., RSS 2025) — The most practical deployment of a physics-informed world model on real robots. Integrates a differentiable physics engine with Gaussian Splatting for end-to-end learning of 3D dynamics from visual observations.

§4 Planner: From World Understanding to Action

4.1 What Is a Planner-Type World Model?

A Planner-type world model outputs actions: given an observation of the current state, what should the agent do next? The world model’s role is to simulate the consequences of candidate actions so that the agent can choose the one that leads to the best outcome. This approach, called model-based reinforcement learning (MBRL), contrasts with model-free RL, where the agent learns to act directly from trial-and-error reward signals without an internal simulation.

Planner-type world models have been developed primarily in two paradigms: learning a compact latent dynamics model and training a policy inside its imagined rollouts (the Dreamer lineage), and using large language or vision-language models to directly reason about and generate actions (the Voyager lineage). These two paradigms represent fundamentally different answers to the question of where the “world knowledge” comes from: learned from environment interaction, or pre-trained from internet-scale data.

4.2 Learning to Act in Imagination

The Dreamer Lineage. Dreamer (Hafner et al., ICLR 2020) introduced the core idea: learn a Recurrent State-Space Model (RSSM) that compresses high-dimensional observations into a compact latent state and predicts how that state evolves. Then train an actor-critic policy entirely within the imagined latent trajectories — the agent never needs to act in the real environment during policy learning. DreamerV2 (ICLR 2021) replaced continuous latent variables with categorical ones, achieving the first human-level performance on the 55-game Atari benchmark by a model-based agent. DreamerV3 (Hafner et al., 2023) scaled this approach to 150+ diverse domains — Atari games, DeepMind Control Suite, and Minecraft — with a single set of hyperparameters. It was the first algorithm to collect diamonds in Minecraft from scratch, a task that requires navigating a complex crafting technology tree over thousands of environment steps. DreamerV3’s RSSM encodes two components: a deterministic recurrent state that summarizes history, and a stochastic state that captures what is unpredictable. The policy and value function are trained entirely on imagined rollouts sampled from this model.

Beyond RSSMs. Several 2025 works have proposed alternative architectures to the RSSM backbone. **IRIS** (Micheli et al., ICLR 2023 Notable Top 5%) first cast world model learning as a sequence modeling problem, using a discrete autoencoder with an autoregressive Transformer to predict future latent states, achieving a mean human-normalized score of 1.046 on Atari 100k. **DIAMOND** (Alonso et al., NeurIPS 2024 Spotlight) replaced discrete latents with a diffusion-based world model operating directly in pixel space, achieving 1.46 on Atari 100k — the best result for agents trained purely within a world model — by preserving visual details that discrete tokenization loses. **TWISTER** (Burchi & Timofte, ICLR 2025) replaced the RNN-based RSSM with a Transformer augmented by Action-Conditioned Contrastive Predictive Coding, achieving 162% on the same benchmark. **DyMoDreamer** (Zhang et al., NeurIPS 2025) added a dynamic modulation mechanism that focuses the world model on task-relevant features using inter-frame differences, reaching 156.6%. **Drama** (Wang et al., ICLR 2025) replaced both RNN and Transformer with a Mamba state-space model backbone, achieving competitive performance with only 7 million parameters. These works suggest that the RSSM — dominant since 2020 — is being reconsidered, with Transformers, diffusion models, and state-space models emerging as viable alternatives.

4.3 LLMs as World Models and Planners

A parallel development starting in 2023 asked a different question: what if a large language model, pre-trained on internet text that describes the world, already contains enough world knowledge to serve as a planner?

Voyager (Wang et al., NeurIPS 2023). Voyager is an LLM-powered agent that plays Minecraft using GPT-4 as its reasoning core. The agent operates through three mechanisms. First, an automatic curriculum proposes progressively harder tasks based on the agent’s current capabilities. Second, a skill library stores executable JavaScript code for each learned behavior, enabling compositional reuse — the agent does not forget how to craft a wooden pickaxe when it moves on to iron. Third, an iterative prompting loop functions as closed-loop planning: the LLM generates a plan, the agent executes it in the game, and environmental feedback (error messages, inventory state) is fed back to the LLM for revision. Voyager collected 3.3 times more unique items and progressed through the technology tree 15.3 times faster than prior methods. Critically, Voyager never learned a dynamics model of Minecraft — its world knowledge came entirely from GPT-4’s pre-training.

Unified Vision-Language-Action Models. OmniJARVIS (Wang et al., NeurIPS 2024) represents

the convergence of vision-language models and planner-type world models. The key innovation is behavior tokenization: long action sequences are compressed into a small number of discrete tokens, which are then appended to the LLM’s vocabulary. A single autoregressive model generates both reasoning text and action tokens in the same forward pass — it thinks and acts within one architecture. This unified approach eliminates the separation between the world model (which predicts what happens) and the planner (which chooses what to do); the same model does both.

4.4 Two Paradigms Compared

The Dreamer lineage and the Voyager lineage offer complementary strengths. Dreamer’s approach — learning a compact latent model from environment interaction — is data-efficient (Atari 100k uses only 2 hours of gameplay) and requires no pre-training. But the learned model is environment-specific; transferring it to a new game requires retraining. Voyager’s approach — using a pre-trained LLM as the world model — generalizes zero-shot to new tasks within the same environment, but depends on a massive pre-trained model with high inference cost and latency. The contrast suggests a natural hybrid: pre-train a general world model, then fine-tune it with a small amount of environment-specific interaction data. Cosmos follows this pattern for rendering; LS-Imagine (Li et al., ICLR 2025 Oral) applies a related idea to Minecraft planning by combining short-term and long-term (“jumpy”) state predictions in a single latent world model.

4.5 Core Papers for This Chapter

1. **DreamerV3** (Hafner et al., 2023) — The most mature and general version of the model-based RL world model lineage. Learns a compact latent dynamics model from pixels and trains a policy entirely in imagination. Masters 150+ domains with fixed hyperparameters and collects diamonds in Minecraft from scratch.
2. **Voyager** (Wang et al., NeurIPS 2023) — The influential demonstration that an LLM can serve as both world model and planner, changing how the field thinks about where world knowledge can come from. Uses GPT-4 with automatic curriculum, skill library, and iterative prompting for open-ended Minecraft exploration.
3. **OmniJARVIS** (Wang et al., NeurIPS 2024) — Unifies vision, language, and action within a single autoregressive model through behavior tokenization. Represents the convergence of VLMs and planner-type world models toward a single architecture that perceives, reasons, and acts.

§5 Open Problems and Future Directions

This section describes four concrete open problems in world model research. For each, we identify the specific barrier that currently blocks progress, propose possible attack vectors for overcoming that barrier, and estimate the difficulty of making meaningful progress.

5.1 Long-Horizon Consistency and Causal Reasoning

Background. When a world model generates a sequence of future states — whether pixels, physical states, or actions — errors accumulate at each step. A small prediction error at step 1 becomes a larger error at step 10 and a catastrophic divergence by step 100. This is the long-horizon consistency problem. It affects all three categories: Renderer videos degrade beyond 30 seconds, Simulator

dynamics diverge beyond roughly 1,000 timesteps, and Planner rollouts become unreliable for tasks requiring extended chains of decisions (such as the Minecraft technology tree, which spans thousands of actions).

Barrier. The fundamental barrier is that current world models learn one-step prediction objectives — “given the current state, predict the next state” — and chain these predictions autoregressively. This is inherently unstable because the model is never trained on its own multi-step outputs. Each step consumes a slightly corrupted input (the previous step’s imperfect output), which pushes the model into regions of state space it never encountered during training. This is technically called distribution shift: the training distribution (real environment states) and the inference distribution (the model’s own generated states) diverge over time.

A deeper issue is that one-step prediction does not require causal understanding. To predict the next frame of a bouncing ball video, a model can interpolate between visually similar frames in its training data without understanding that the ball follows a parabolic trajectory determined by gravity and initial velocity. The model learns correlations, not causal mechanisms. Kang et al. (ICML 2025) provided systematic evidence for this: video models presented with out-of-distribution physical scenarios (e.g., a ball bouncing in reversed gravity) continued to generate “plausible-looking” videos that violated the underlying physics, suggesting they were retrieving nearest-neighbor training examples rather than applying general physical principles.

Attack Vectors. Three directions appear promising:

- (a) *Causal training objectives.* Diffusion Forcing (Chen et al., NeurIPS 2024) introduced a training paradigm where each token in a sequence receives an independent noise level, enabling the model to learn both generation (high noise to clean) and causal conditioning (clean past to noisy future) within a single framework. Extending this idea to longer horizons and to simulator architectures is a natural next step. Training objectives that explicitly penalize multi-step divergence — rather than only single-step error — could force models to learn stable dynamics.
- (b) *Hierarchical temporal abstraction.* Rather than predicting every intermediate frame or state, a world model could operate at multiple timescales: a high-level planner proposes subgoals (“reach the tree”), a mid-level model generates a coarse trajectory, and a low-level model fills in the details. LS-Imagine (Li et al., ICLR 2025 Oral) demonstrated this idea for Minecraft by combining short-term and long-term (“jumpy”) latent state predictions. Extending hierarchical architectures to the Renderer and Simulator categories is largely unexplored.
- (c) *External memory.* WorldMem (NeurIPS 2025) showed that augmenting a video generation model with an external memory module — a compact store of previously generated states that the model can query during generation — significantly improves long-term consistency. Memory-augmented architectures have a longer history in RL (JARVIS-1, T-PAMI 2025, used RAG over past planning experiences), but their application to learned physics simulation is new and largely untested at scale.

Difficulty: Medium-High. The autoregressive error accumulation problem has been studied for years in sequence modeling. The newer challenge is combining architectural innovations (causal objectives, hierarchical planning, external memory) with the scale needed for general-purpose world models. Progress on any one of the three attack vectors above would constitute a meaningful advance.

5.2 Unification of Renderer, Simulator, and Planner

Background. Current world models are built within one category: a Renderer produces pixels, a Simulator produces physical states, a Planner produces actions. But a general-purpose world model would need to do all three — render scenes, simulate physics, and plan actions — depending on the downstream task. This is the unification problem.

Barrier. The internal representations of the three categories are incompatible. A Renderer’s latent space encodes pixel-level appearance patterns; a Simulator’s latent space encodes physical quantities (mass, position, velocity); a Planner’s latent space encodes action-value relationships. There is no shared vocabulary across these representations. You cannot feed the output of a Simulator into a Planner without manually designing an interface (e.g., extracting physical state variables and formatting them as planner inputs), and you cannot feed Renderer latents into a Simulator at all — they encode different types of information.

A second barrier is architectural. Renderers use diffusion models or autoregressive transformers over visual tokens. Simulators use graph networks or state-space models over physical variables. Planners use RSSMs or LLM-based sequence models over latent states or text tokens. These architectures were designed independently for their respective domains and do not compose naturally.

Attack Vectors.

- (a) *Unified tokenization.* The core idea is to map all three output types — pixels, physical states, actions — into a shared token space. OmniJARVIS demonstrated this in one direction: it tokenized actions into discrete behavior tokens that could be interleaved with text tokens in a single autoregressive model. Extending this approach to also include physical state tokens (for simulation) and pixel tokens (for rendering) would create a model that can generate any of the three output types from the same latent representation. The technical challenge is that pixel tokens are vastly higher-dimensional than action or state tokens; some form of hierarchical or compressed pixel representation would be needed.
- (b) *Causal bridges between categories.* Vid2World (Huang et al., 2025) demonstrated that a pre-trained video diffusion model (a Renderer) can be converted into an action-conditioned interactive simulator by modifying the architecture to be causal (past frames condition future frames) and adding action conditioning. This suggests a general strategy: rather than building a unified model from scratch, build bridges between existing category-specific models. A Renderer-to-Simulator bridge would extract physically accurate state representations from video generation models. A Simulator-to-Planner bridge would enable policies trained in learned simulations to transfer to real environments.
- (c) *The Simulator as intermediate representation.* Li’s original observation suggests a specific path to unification: if a model masters simulation (producing physically accurate state representations), rendering and planning become downstream tasks built on top of that representation. A Simulator-first architecture would learn a shared physical state representation, then attach a Renderer head (for visual output) and a Planner head (for action output) as needed. This architecture has not been attempted at scale.

Difficulty: High. This requires reconciling three largely independent research traditions. The most feasible near-term target is a two-category bridge (Renderer-to-Simulator or Simulator-to-Planner) rather than full three-way unification.

5.3 Data Efficiency vs. Generalization Trade-off

Background. There exists a tension between two desirable properties of a world model: data efficiency (learning from limited environment-specific experience) and generalization (applying to new tasks or environments without retraining). The Dreamer lineage is data-efficient — DreamerV3 learns Atari games from 100,000 environment frames — but each new game requires training a new model. The Voyager lineage generalizes across tasks within Minecraft using a single pre-trained LLM, but the LLM itself required training on trillions of tokens of internet data, and its inference cost per action is orders of magnitude higher than DreamerV3’s policy network.

Barrier. The efficiency-generalization trade-off is not a superficial engineering issue. It reflects a fundamental question about where world knowledge should reside. In the Dreamer paradigm, world knowledge is extracted from environment interaction — the model learns physics by observing pixels change in response to actions. This is sample-efficient for a specific environment but does not transfer. In the Voyager paradigm, world knowledge is pre-loaded from internet-scale text and image data — the model has read about Minecraft before it ever plays. This transfers well but incurs the pre-training cost and the inference cost of a large model.

Attack Vectors.

- (a) *Pre-trained world models with environment-specific fine-tuning.* The Cosmos platform follows this pattern for rendering: a large model is pre-trained on diverse video data, then fine-tuned on domain-specific footage. For Planners, ROCKET-1 (Cai et al., CVPR 2025) demonstrated a hybrid architecture: a large VLM handles high-level task decomposition and spatial reasoning, while a small TransformerXL policy (trained on environment-specific data) handles low-level motor control. This decoupling allows the expensive general-purpose model to operate at a coarse timescale while the cheap specialized model handles fine-grained execution.
- (b) *World model distillation.* An alternative is to use an LLM-based planner to generate training data for a smaller, environment-specific world model and policy. The LLM explores the environment, generates successful trajectories, and these trajectories are used to train a compact Dreamer-style model that runs efficiently at inference time. This approach has been explored in robotics (DayDreamer showed that Dreamer-style models can learn on physical robots with limited data) but not systematically for game or simulation environments.
- (c) *Meta-learning across environments.* Rather than pre-training on internet data or training from scratch on each environment, a world model could be meta-trained across many environments to learn a dynamics prior that adapts quickly to new environments with minimal data. DreamerV3’s fixed-hyperparameter results across 150+ domains hint at this possibility. The model architecture and learning algorithm generalize; what does not generalize is the learned latent dynamics, which are environment-specific.

Difficulty: Medium. This is primarily an engineering and resource-allocation problem rather than a fundamental conceptual barrier. The key open question is whether a hybrid architecture (large pre-trained planner + small learned dynamics model) can match or exceed the performance of either pure approach.

5.4 Multi-Modal World Modeling Beyond Vision

Background. Nearly all current world models operate on visual data — RGB images or video frames. Human understanding of the physical world, in contrast, is multi-modal: we integrate

vision, sound, touch, and proprioception (the sense of our own body’s position and movement). A world model that only sees is missing information that is often critical for understanding physical interactions. The sound of a collision conveys material properties (metal vs. plastic). Tactile feedback conveys surface texture and contact force. These signals are available in real-world interaction but absent from current world model training pipelines.

Barrier. The primary barrier is data. Multi-modal synchronized data — video with aligned audio, force/torque readings, and tactile sensor streams — is scarce compared to video-only data. Internet video (YouTube, TikTok) provides an effectively unlimited supply of visual data, but the audio track is often irrelevant (music, voiceover, noise) and tactile data is non-existent. Multi-modal data collection requires instrumented physical environments (robots with force sensors, tactile skins, microphones), which is expensive and does not scale to internet-sized datasets.

A second barrier is architectural. Current world model architectures are designed for visual input (2D CNNs, ViTs, 3D convolutions for video). Integrating additional modalities requires designing separate encoders for each modality and, more challenging, learning a shared representation where visual, auditory, and tactile features are aligned in a common space. Cross-modal alignment — ensuring that the visual appearance of a collision and the sound of that collision map to the same underlying event representation — is an unsolved representation learning problem.

Attack Vectors.

- (a) *Game environments as multi-modal testbeds.* Minecraft includes both visual rendering and spatialized audio (footsteps, block breaking, animal sounds). These audio signals are generated by the game engine and are perfectly synchronized with visual events. Minecraft could serve as a controlled testbed for multi-modal world modeling: a model trained to predict both the next video frame and the next audio spectrogram from the current state would learn a joint visual-auditory representation without requiring expensive physical data collection. No current world model, to our knowledge, uses audio from game environments.
- (b) *Synthetic data from physics simulators.* Differentiable physics simulators (used in the Simulator category) can generate not only visual renderings but also contact forces, collision events, and material deformation data. These simulators could be instrumented to output synthetic audio (by simulating the acoustic consequences of physical events) and synthetic tactile readings (by recording contact forces at interaction points). The resulting multi-modal dataset, while synthetic, would be perfectly aligned and unlimited in scale. The open question is whether models trained on synthetic multi-modal data transfer to real-world multi-modal perception.
- (c) *Pre-trained modality encoders with late fusion.* An architecturally simpler approach is to use separate pre-trained encoders for each modality — a vision transformer for images, an audio spectrogram transformer for sound, a force/torque encoder for touch — and fuse their representations late (at a high level of abstraction) rather than early (at the raw input level). This avoids the need for joint multi-modal pre-training but may miss fine-grained cross-modal correspondences.

Difficulty: Medium. The architectural components exist (pre-trained encoders for vision, audio, and increasingly for touch). The bottleneck is data: building even a modest multi-modal world model dataset with aligned vision, audio, and tactile streams would be a significant engineering contribution. Game environments offer the most practical near-term path.

References

[References to be compiled from the merged literature database. All citations verified against arXiv, Semantic Scholar, OpenAlex, and Crossref.]

Core Papers by Chapter

§1 Introduction - Ha, D. & Schmidhuber, J. (2018). Recurrent World Models Facilitate Policy Evolution. NeurIPS 2018. arXiv:1803.10122. - Li, F. (2026). World Model Taxonomy. X post, June 4, 2026. - Zhu, Z. et al. (2024). Is Sora a World Simulator? A Comprehensive Survey on General World Models and Beyond. arXiv:2405.03520. - Ding, J. et al. (2025). Understanding World or Predicting Future? A Comprehensive Survey of World Models. ACM Computing Surveys. arXiv:2411.14499. - Kong, L. et al. (2025). 3D and 4D World Modeling: A Survey. arXiv:2509.07996.

§2 Renderer - Brooks, T., Peebles, B. et al. (2024). Video Generation Models as World Simulators. OpenAI Technical Report. - Bruce, J. et al. (2024). Genie: Generative Interactive Environments. ICML 2024 Best Paper. arXiv:2402.15391. - NVIDIA Cosmos Team (2025). Cosmos World Foundation Model Platform for Physical AI. arXiv:2501.03575. - Feng, R. et al. (2024). The Matrix: Infinite-Horizon World Generation with Real-Time Moving Control. arXiv:2412.03568. - Valevski, D. et al. (2025). GameNGen: Diffusion Models Are Real-Time Game Engines. ICLR 2025. arXiv:2408.14837. - Ren, X. et al. (2025). GEN3C: 3D-Informed World-Consistent Video Generation. CVPR 2025 Highlight. arXiv:2503.03751. - Kang, B. et al. (2025). How Far Is Video Generation from World Model: A Physical Law Perspective. ICML 2025. arXiv:2411.02385. - Duan, H. et al. (2025). WorldScore: A Unified Evaluation Benchmark for World Generation. ICCV 2025. arXiv:2504.00983. - Chen, B. et al. (2024). Diffusion Forcing. NeurIPS 2024. arXiv:2407.01392.

§3 Simulator - Sanchez-Gonzalez, A. et al. (2020). Learning to Simulate Complex Physics with Graph Networks. ICML 2020. arXiv:2002.09405. - Zhu, Y. et al. (2025). One-Shot Real-to-Sim via End-to-End Differentiable Simulation and Rendering. RA-L 2025. arXiv:2412.00259. - Li, W. et al. (2025). PIN-WM: Learning Physics-Informed World Models. RSS 2025. arXiv:2504.16693. - Pfaff, T. et al. (2021). Learning Mesh-Based Simulation with Graph Networks. ICLR 2021 Spotlight. arXiv:2010.03409. - Mosbach, M. et al. (2025). SOLD: Slot Object-Centric Latent Dynamics Models. ICML 2025. arXiv:2410.08822. - Wang, M. et al. (2026). ContactGaussian-WM: Learning Physics-Grounded World Model from Videos. arXiv:2602.11021. - Lillemark, H. et al. (2025). FloWM: Flow Equivariant World Models. NeurIPS 2025. arXiv:2601.01075.

§4 Planner - Hafner, D. et al. (2023). Mastering Diverse Domains through World Models (DreamerV3). arXiv:2301.04104. - Wang, G. et al. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. NeurIPS 2023. arXiv:2305.16291. - Wang, Z. et al. (2024). OmniJARVIS: Unified Vision-Language-Action Tokenization. NeurIPS 2024. arXiv:2407.00114. - Hafner, D. et al. (2020). Dream to Control (Dreamer). ICLR 2020. arXiv:1912.01603. - Hafner, D. et al. (2021). Mastering Atari with Discrete World Models (DreamerV2). ICLR 2021. arXiv:2010.02193. - Burchi, M. & Timofte, R. (2025). TWISTER: Learning Transformer-based World Models with CPC. ICLR 2025. arXiv:2503.04416. - Zhang, B. et al. (2025). DyMoDreamer: World Modeling with Dynamic Modulation. NeurIPS 2025. arXiv:2509.24804. - Li, J. et al. (2025). LS-Imagine: Open-World RL over Long Short-Term Imagination. ICLR 2025 Oral. arXiv:2410.03618. - Cai, S. et al. (2025). ROCKET-1: Master Open-World Interaction. CVPR 2025. arXiv:2410.17856. - Micheli, V. et al. (2023). IRIS: Transformers are Sample-Efficient World Models. ICLR 2023. arXiv:2209.00588. - Alonso, E. et al. (2024). DIAMOND: Diffusion for World Modeling. NeurIPS 2024. arXiv:2405.12399. - Wang, S. et al. (2024). EfficientZero V2: Mastering Discrete and

Continuous Control with Limited Data. ICML 2024 Spotlight. arXiv:2403.00564.

§5 Open Problems - Chen, B. et al. (2024). Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion. NeurIPS 2024. arXiv:2407.01392. - Li, J. et al. (2025). LS-Imagine: Open-World RL over Long Short-Term Imagination. ICLR 2025 Oral. arXiv:2410.03618. - Xiao, Z. et al. (2025). WorldMem: Long-term Consistent World Simulation with Memory. NeurIPS 2025. arXiv:2504.12369. - Wang, Z. et al. (2025). JARVIS-1: Open-world Multi-task Agents with Memory-Augmented Multimodal Language Models. IEEE T-PAMI, 47(3). arXiv:2311.05997. - Huang, S. et al. (2025). Vid2World: Crafting Video Diffusion Models to Interactive World Models. arXiv:2505.14357. - Cai, S. et al. (2025). ROCKET-1: Master Open-World Interaction with Visual-Temporal Context Prompting. CVPR 2025. arXiv:2410.17856. - Wu, P. et al. (2022). DayDreamer: World Models for Physical Robot Learning. CoRL 2022. arXiv:2206.14176.

This survey was compiled in June 2026. All cited works have verifiable arXiv identifiers, DOIs, or peer-reviewed venue publications. We gratefully acknowledge the authors of the cited works and the maintainers of the open-source world model platforms that made this survey possible.